CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA

ETEC PROFESSOR EDSON GALVÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

GIOVANA TOBIAS RIBEIRO

MARCOS VINICIUS DEMÉTRIO

NICOLLAS JOSÉ ROCHEL

SISTEMA DE MONITORAMENTO DE SUÍNOS

Protótipo de rastreamento de entradas e saídas com Arduino e aplicação web

ITAPETININGA

2026

GIOVANA TOBIAS RIBEIRO
MARCOS VINICIUS DEMÉTRIO
NICOLLAS JOSÉ ROCHEL

SISTEMA DE MONITORAMENTO DE SUÍNOS

Protótipo de rastreamento de entradas e saídas com Arduino e aplicação web

Trabalho de Conclusão de Curso (TCC) apresentado ao Conselho de Curso de Desenvolvimento de Sistemas, da Escola Técnica Estadual Professor Edson Galvão, para obtenção do título de Técnico em Desenvolvimento de Sistemas.

Orientadora: Prof.ª Carolina Haubt Salvador
Coorientadora: Prof.ª Sônia Maria Cardoso

ITAPETININGA
2026

GIOVANA TOBIAS RIBEIRO
MARCOS VINICIUS DEMÉTRIO
NICOLLAS JOSÉ ROCHEL

SISTEMA DE MONITORAMENTO DE SUÍNOS

Protótipo de rastreamento de entradas e saídas com Arduino e aplicação web

Trabalho de Conclusão de Curso (TCC) apresentado no Curso de Desenvolvimento de Sistemas, da Escola Técnica Estadual Prof. Edson Galvão, para obtenção do título de Técnico em Desenvolvimento de Sistemas.

Banca Examinadora

Prof.ª Carolina Haubt Salvador
Etec Professor Edson Galvão
Orientadora

[PREENCHER — NOME DO EXAMINADOR]
[PREENCHER — INSTITUIÇÃO]
Examinador(a)

[PREENCHER — NOME DO EXAMINADOR]
[PREENCHER — INSTITUIÇÃO]
Examinador(a)

Itapetininga, ____ de _____ de 2026.

DEDICATÓRIA

Dedicamos este trabalho, primeiramente, a Deus, por nos conceder força, sabedoria e perseverança para chegarmos até aqui.

Aos nossos pais e familiares, pelo amor, apoio, compreensão e incentivo ao longo de toda esta caminhada. Nos momentos de dificuldade, foram suas palavras de carinho e encorajamento que nos deram forças para continuar.

Aos nossos amigos, professores e a todos que, de alguma forma, contribuíram para nossa formação e estiveram ao nosso lado durante essa jornada, tornando os desafios mais leves e os momentos de conquista ainda mais especiais.

Por fim, dedicamos este trabalho a nós mesmos, pela união, parceria, dedicação e por não desistirmos diante das dificuldades encontradas ao longo do caminho. Esta conquista representa o resultado de todo o esforço e aprendizado que compartilhamos e marca o encerramento de uma importante etapa de nossas vidas.

"Àquela que, com paciência e amor de Mãe, desatou os nós invisíveis do cansaço e das incertezas, permitindo-me alcançar esta vitória." – *Frase inspirada em Nossa Senhora Desatadora de Nós*

RESUMO

A suinocultura brasileira opera hoje em escala industrial e sob exigências crescentes de rastreabilidade, mas o controle da movimentação de animais entre baias ainda é feito, em muitas propriedades, por anotação manual. Este trabalho apresenta o desenvolvimento do SuinoTech, um protótipo funcional de sistema de monitoramento de entradas e saídas de suínos que integra um microcontrolador Arduino a uma aplicação web. O objetivo foi demonstrar, em escala reduzida e com componentes de baixo custo, que a captura automática desses eventos e sua disponibilização imediata ao produtor são tecnicamente viáveis. A pesquisa caracteriza-se como aplicada, de natureza exploratória, conduzida por prototipagem incremental. Na camada física, um Arduino Uno monitora oito sensores digitais correspondentes às entradas e saídas de quatro baias, aplica tratamento de ruído por debounce e transmite cada evento pela porta serial em formato JSON. A comunicação com o servidor é intermediada pelo próprio navegador, por meio da Web Serial API, dispensando bibliotecas nativas e configuração manual de porta. O servidor, construído em Node.js com Express, valida e persiste os registros e os distribui em tempo real por WebSocket, enquanto a interface web apresenta um painel de ocupação por baia, alertas automáticos de capacidade, histórico com filtros e exportação de relatórios em CSV. Os testes funcionais realizados em ambiente controlado confirmaram a integridade dos dados entre o acionamento do sensor e sua exibição na interface, validando a arquitetura proposta como base para uma futura versão com sensores reais e comunicação sem fio.

Palavras-chave: Suinocultura; Internet das coisas; Arduino; Sistemas embarcados; Monitoramento em tempo real.

ABSTRACT

Brazilian pig farming operates on an industrial scale and faces growing traceability requirements, yet the movement of animals between pens is still recorded manually on many farms. This paper presents the development of SuinoTech, a working prototype for monitoring pig entries and exits that integrates an Arduino microcontroller with a web application. The goal was to demonstrate, on a reduced scale and with low-cost components, that automatic capture of these events and their immediate delivery to the producer are technically feasible. The research is applied and exploratory in nature and was conducted through incremental prototyping. In the physical layer, an Arduino Uno monitors eight digital sensors corresponding to the entries and exits of four pens, applies debounce filtering and transmits each event over the serial port in JSON format. Communication with the server is handled by the browser itself through the Web Serial API, eliminating native libraries and manual port configuration. The server, built with Node.js and Express, validates and persists the records and broadcasts them in real time over WebSocket, while the web interface displays pen occupancy, automatic capacity alerts, a filterable history and CSV report export. Functional tests carried out in a controlled environment confirmed data integrity between sensor activation and interface display, validating the proposed architecture as a basis for a future version with real sensors and wireless communication.

Keywords: pig farming; internet of things; Arduino; embedded systems; real-time monitoring.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura em camadas do sistema SuinoTech 31
Figura 2 – Fluxo de um evento, do sensor à interface do produtor 32
Figura 3 – Diagrama de casos de uso do sistema 36
Figura 4 – Mapeamento de pinos e esquema de ligação do Arduino 37
Figura 5 – Protótipo montado em protoboard 38
Figura 6 – Modelo de dados do sistema 42
Figura 7 – Interface do dashboard em tempo real 45
Figura 8 – Interface de histórico e relatórios 46

Quadro 1 – Etapas do desenvolvimento e respectivos produtos 26
Quadro 2 – Materiais de hardware utilizados no protótipo 27
Quadro 3 – Recursos de software utilizados 28
Quadro 4 – Cronograma de execução do projeto 30
Quadro 5 – Requisitos funcionais do sistema 33
Quadro 6 – Requisitos não funcionais do sistema 34
Quadro 7 – Regras de negócio implementadas 35
Quadro 8 – Mapeamento de pinos do microcontrolador 37
Quadro 9 – Endpoints da API REST 41
Quadro 10 – Eventos de comunicação em tempo real 44
Quadro 11 – Roteiro de testes funcionais executados 46

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ABPA	Associação Brasileira de Proteína Animal
API	Application Programming Interface (interface de programação de aplicações)
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
Embrapa	Empresa Brasileira de Pesquisa Agropecuária
Etec	Escola Técnica Estadual
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment (ambiente integrado de desenvolvimento)
IoT	Internet of Things (internet das coisas)
IR	Infrared (infravermelho)
JSON	JavaScript Object Notation
LED	Light Emitting Diode (diodo emissor de luz)
REST	Representational State Transfer
SGAS	Sistema de Gestão Ambiental da Suinocultura
TCC	Trabalho de Conclusão de Curso
USB	Universal Serial Bus

LISTA DE SÍMBOLOS

ms	milissegundo
V	volt
Ω	ohm
baud	unidade de taxa de sinalização em transmissão serial
%	por cento

SUMÁRIO

1 Introdução 13
1.1 Contextualização 13
1.2 Problema de pesquisa 14
1.3 Estrutura do trabalho 15
2 Justificativa 16
3 Objetivos 17
3.1 Objetivo geral 17
3.2 Objetivos específicos 17
4 Referencial teórico 18
4.1 A suinocultura brasileira e a gestão do plantel 18
4.2 Internet das Coisas aplicada à produção animal 19
4.3 Sistemas embarcados e microcontroladores 20
4.3.1 A plataforma Arduino 20
4.3.2 Sensores digitais e tratamento de ruído 21
4.4 Comunicação serial e a Web Serial API 22
4.5 Aplicações web e arquitetura cliente-servidor 22
4.5.1 O estilo arquitetural REST 23
4.5.2 Comunicação bidirecional com WebSocket 23
4.6 Persistência de dados 24
4.7 Processo de desenvolvimento por prototipagem 24
5 Materiais e métodos 26
5.1 Caracterização da pesquisa 26
5.2 Etapas do desenvolvimento 26
5.3 Materiais e recursos utilizados 27
5.3.1 Recursos de hardware 27
5.3.2 Recursos de software 28
5.4 Procedimentos de validação 29
5.5 Cronograma de execução 30
6 Desenvolvimento do sistema e resultados 31
6.1 Visão geral e arquitetura da solução 31
6.2 Especificação de requisitos 33
6.2.1 Requisitos funcionais 33
6.2.2 Requisitos não funcionais 34
6.2.3 Regras de negócio 35
6.3 Casos de uso 36
6.4 Camada física: hardware e firmware 37
6.4.1 Montagem e mapeamento de pinos 37
6.4.2 Firmware do microcontrolador 38

6.5 Camada de comunicação 39
6.6 Camada de servidor 41
6.6.1 Estrutura e organização do backend 41
6.6.2 A API REST 41
6.6.3 Persistência e modelo de dados 42
6.6.4 Distribuição em tempo real 44
6.7 Camada de apresentação 44
6.7.1 Dashboard em tempo real 45
6.7.2 Histórico e relatórios 46
6.8 Testes e validação 46
6.9 Discussão dos resultados 48
6.10 Limitações do protótipo 49
7 Considerações finais 51
7.1 Trabalhos futuros 52
REFERÊNCIAS 54
APÊNDICE A – Trechos comentados do código-fonte 57
APÊNDICE B – Roteiro detalhado de testes funcionais 59

1 Introdução

A produção de suínos ocupa posição estratégica no agronegócio brasileiro. Segundo dados consolidados pela Associação Brasileira de Proteína Animal (2026), o país produziu 5,592 milhões de toneladas de carne suína em 2025, das quais 72,99% foram destinadas ao mercado interno e 27,01% à exportação, o que colocou o Brasil na terceira posição entre os maiores exportadores mundiais e na quarta entre os maiores produtores. O consumo interno per capita atingiu 19,1 quilos por habitante ao ano no mesmo período.

Esse desempenho, contudo, convive com uma exigência crescente de controle sobre o processo produtivo. A Empresa Brasileira de Pesquisa Agropecuária (2024b) firmou parceria com a própria ABPA[1] para definir protocolos nacionais de rastreabilidade para as cadeias de suínos e frangos de corte, em resposta a legislações nacionais e a requisitos de mercados importadores. A rastreabilidade deixou de ser um diferencial comercial e passou a integrar as condições de acesso a mercados.

Entre os dados que sustentam qualquer sistema de rastreabilidade está o registro da movimentação dos animais: quantos entraram em determinada baia, quantos saíram e em que momento. É justamente nesse ponto que muitas propriedades ainda dependem de anotação em papel ou de planilhas preenchidas ao fim do expediente. O presente trabalho parte dessa lacuna e propõe um protótipo que automatiza a captura desses eventos e os disponibiliza ao produtor em tempo real.

1.1 Contextualização

A digitalização da suinocultura brasileira avança de forma desigual. De um lado, há soluções consolidadas em granjas de grande porte: a Empresa Brasileira de Pesquisa Agropecuária (2024a) desenvolveu, em parceria com a iniciativa privada, robôs de distribuição de ração que já operam em escala comercial, e mantém sistemas como o SGAS[2], voltado à gestão ambiental e ao licenciamento de granjas. De outro, permanece um contingente expressivo de produtores para os quais o custo de aquisição e a complexidade de instalação desses sistemas são proibitivos.

[1] [Associação Brasileira de Proteína Animal]
[2] [Sistema de Gestão Ambiental da Suinocultura]

Essa assimetria é reconhecida na literatura sobre tecnologias de precisão aplicadas à produção animal, que aponta o custo de implementação e a precariedade da infraestrutura de conectividade em regiões produtoras como barreiras persistentes, especialmente para pequenos produtores. O problema, portanto, não é a inexistência de tecnologia, mas a distância entre a tecnologia disponível e a realidade operacional de parte relevante do setor.

A Internet das Coisas oferece um caminho para reduzir essa distância. O conceito, sistematizado por Atzori, Iera e Morabito (2010), descreve a integração de objetos físicos dotados de capacidade de identificação, sensoriamento e comunicação a redes de dados, de modo que informações do mundo físico se tornem acessíveis a sistemas computacionais. Aplicado ao ambiente de uma granja, isso significa que um evento físico — a passagem de um animal por uma porteira — pode ser convertido em um registro digital no instante em que ocorre, sem intervenção humana.

Plataformas de prototipagem eletrônica de baixo custo tornaram esse tipo de solução acessível a contextos com pouco recurso. O Arduino, descrito por McRoberts (2015) como uma plataforma de hardware e software livre destinada a facilitar a construção de dispositivos capazes de interagir com o ambiente, permite que sensores simples sejam integrados a sistemas de informação com investimento reduzido e sem exigir formação especializada em eletrônica.

1.2 Problema de pesquisa

A observação da rotina de controle de plantel em propriedades que não dispõem de automação permite identificar três limitações que se reforçam mutuamente.

A primeira é a dependência do registro manual. Quando a contagem de animais que entram e saem de uma baia é anotada por um operador, o dado fica sujeito a esquecimento, a erro de transcrição e a atraso entre o evento e seu registro. Não existe mecanismo que garanta que toda movimentação foi de fato registrada.

A segunda é a ausência de histórico consultável. Registros dispersos em cadernos ou planilhas locais dificultam a reconstituição da movimentação de uma baia em um período determinado, informação que se torna crítica em situações de investigação sanitária ou de auditoria de rastreabilidade.

A terceira é a inexistência de visibilidade imediata. O produtor só toma conhecimento da situação de ocupação das baias quando consulta fisicamente as anotações, o que impede que problemas como a superlotação de uma baia sejam percebidos no momento em que se configuram.

Diante disso, formula-se a seguinte questão de pesquisa: é tecnicamente viável construir, com componentes eletrônicos de baixo custo e tecnologias web de código aberto, um sistema capaz de capturar automaticamente a movimentação de suínos entre baias e disponibilizar essa informação ao produtor em tempo real?

1.3 Estrutura do trabalho

Este trabalho está organizado em cinco seções. A primeira apresenta o contexto, o problema, a justificativa e os objetivos da pesquisa. A segunda reúne o referencial teórico que fundamenta as decisões de projeto, abordando a suinocultura, a Internet das Coisas, os sistemas embarcados, a comunicação serial, a arquitetura de aplicações web e a persistência de dados. A terceira descreve os materiais e os métodos empregados, incluindo a caracterização da pesquisa, as etapas de desenvolvimento e os procedimentos de validação. A quarta detalha a construção do sistema, da especificação de requisitos aos resultados obtidos, e discute as limitações identificadas. A quinta apresenta as considerações finais e as perspectivas de continuidade.

2 Justificativa

A justificativa deste trabalho se apoia em três dimensões: técnica, acadêmica e social.

Do ponto de vista técnico, a proposta demonstra que a integração entre hardware embarcado e aplicação web pode ser obtida sem infraestrutura sofisticada. A solução desenvolvida dispensa servidores em nuvem, contratos de conectividade celular e bibliotecas de compilação nativa, requisitos que costumam inviabilizar projetos semelhantes em ambientes com recursos limitados. Ao concentrar a comunicação com o microcontrolador no próprio navegador, por meio da Web Serial API[3], elimina-se inclusive a necessidade de configuração manual de porta serial, etapa que frequentemente representa a primeira barreira de instalação.

Do ponto de vista acadêmico, o projeto articula conteúdos de diferentes componentes curriculares do curso Técnico em Desenvolvimento de Sistemas — programação embarcada, desenvolvimento web, modelagem de dados, engenharia de software e teste de sistemas — em um artefato único e funcional, o que permite avaliar a aplicação prática desses conhecimentos de forma integrada.

Do ponto de vista social, o trabalho endereça uma demanda concreta do setor produtivo. Se a rastreabilidade se consolida como condição de acesso a mercados, conforme apontado pela Empresa Brasileira de Pesquisa Agropecuária (2024b), soluções de baixo custo que produzam registros digitais confiáveis contribuem para que produtores de menor porte não sejam excluídos desse movimento.

[3] [Application Programming Interface (interface de programação de aplicações)]

3 Objetivos

3.1 Objetivo geral

Desenvolver e validar um protótipo funcional de sistema de monitoramento de entradas e saídas de suínos, integrando um microcontrolador Arduino a uma aplicação web capaz de registrar, armazenar e apresentar os eventos capturados em tempo real.

3.2 Objetivos específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

a) levantar os fundamentos teóricos relativos à Internet das Coisas, a sistemas embarcados, à comunicação serial e ao desenvolvimento de aplicações web em tempo real;

b) especificar os requisitos funcionais, não funcionais e as regras de negócio do sistema de monitoramento;

c) implementar o firmware do microcontrolador responsável pela leitura dos sensores digitais, pelo tratamento de ruído de acionamento e pela transmissão dos eventos;

d) implementar um servidor de aplicação responsável por validar, persistir e distribuir os eventos recebidos;

e) desenvolver uma interface web que apresente a ocupação das baias, os alertas de capacidade, o histórico filtrável dos eventos e a exportação de relatórios;

f) executar testes funcionais que verifiquem a integridade dos dados entre o acionamento do sensor e a apresentação da informação na interface.

4 Referencial teórico

Esta seção reúne os conceitos que sustentam as decisões técnicas adotadas no desenvolvimento do protótipo. A exposição segue a ordem das camadas do sistema construído: parte do domínio de aplicação, a suinocultura, avança para os fundamentos de captura de dados no mundo físico e conclui com os elementos de arquitetura de software empregados no tratamento e na apresentação dessas informações.

4.1 A suinocultura brasileira e a gestão do plantel

A suinocultura brasileira é uma atividade de escala industrial. Os dados consolidados pela Associação Brasileira de Proteína Animal (2026) referentes ao ano de 2025 registram produção de 5,592 milhões de toneladas de carne suína e exportações de 1,510 milhão de toneladas, com receita superior a três bilhões de dólares. A parcela destinada ao abastecimento interno correspondeu a 72,99% do volume produzido, e o consumo per capita alcançou 19,1 quilos por habitante ao ano.

A estrutura de custos da atividade é fortemente determinada pela alimentação animal, baseada em milho e farelo de soja, o que expõe o produtor à volatilidade do mercado de grãos. Em um cenário em que a maior parte do custo é determinada por fatores externos, a margem de ganho do produtor depende fortemente da eficiência com que ele administra os fatores que estão sob seu controle — entre eles, a ocupação das instalações e o acompanhamento do lote.

A gestão do plantel compreende o conjunto de práticas de acompanhamento dos animais ao longo do ciclo produtivo. A Empresa Brasileira de Pesquisa Agropecuária (2023) inclui entre os elementos essenciais desse manejo as práticas de alimentação, biosseguridade, bem-estar animal e rastreabilidade. Cada um desses elementos depende, em alguma medida, de informação confiável sobre onde os animais estão e quando se moveram.

O tema da rastreabilidade ganhou densidade institucional recente. A Empresa Brasileira de Pesquisa Agropecuária (2024b) firmou acordo com a Associação Brasileira de Proteína Animal para desenvolver, em até três anos, protocolos de produção sustentável e rastreabilidade para as cadeias de suínos e frangos de corte, tomando como base o Sistema Brasileiro de Agrorrastreabilidade. A justificativa

apresentada é o atendimento a legislações nacionais e a exigências de mercados importadores. Registra-se, no mesmo documento, que embora muitas empresas do setor já disponham de sistemas próprios, a proposta visa padronizar as ações em um programa de alcance nacional.

Ferramentas digitais desenvolvidas pela própria Embrapa[4] ilustram a direção desse movimento. O Sistema de Gestão Ambiental da Suinocultura automatiza análises antes conduzidas manualmente ou por meio de planilhas, com o objetivo declarado de trazer confiabilidade e rastreabilidade ao monitoramento ambiental da suinocultura nacional (Empresa Brasileira de Pesquisa Agropecuária, 2020). Mais recentemente, o sistema GeoFert passou a registrar de forma digital operações antes anotadas manualmente no manejo de dejetos suínos (Empresa Brasileira de Pesquisa Agropecuária, 2026).

O padrão que emerge desses casos é consistente: substituir o registro manual pelo registro digital automático é o passo inicial de qualquer estratégia de rastreabilidade. É precisamente esse passo, aplicado à movimentação de animais entre baias, que o presente trabalho procura demonstrar em escala de protótipo.

4.2 Internet das Coisas aplicada à produção animal

Atzori, Iera e Morabito (2010) definem a Internet das Coisas como um paradigma no qual objetos físicos do cotidiano, dotados de capacidade de identificação, sensoriamento e comunicação, passam a integrar redes de dados, tornando-se fontes de informação para sistemas computacionais. Os autores destacam que a viabilidade do paradigma depende da convergência entre tecnologias de identificação, redes de sensores e middleware capaz de tratar os dados coletados.

Essa definição delimita três funções que qualquer aplicação de Internet das Coisas precisa cumprir: perceber um fenômeno no mundo físico, transmiti-lo por uma rede e processá-lo em um sistema capaz de transformá-lo em informação útil. No protótipo desenvolvido, essas três funções correspondem, respectivamente, aos sensores digitais instalados nas baias, à transmissão serial dos eventos e ao servidor de aplicação que os interpreta e os apresenta.

No contexto da produção animal, a aplicação desses conceitos é usualmente designada como zootecnia ou pecuária de precisão. A literatura da área descreve o

[4] [Empresa Brasileira de Pesquisa Agropecuária]

emprego de sensores, atuadores e sistemas de automação para monitoramento de variáveis ambientais e de desempenho, com finalidade de garantir conforto térmico, bem-estar animal e ganho de produtividade. Os mesmos estudos, entretanto, apontam como obstáculos recorrentes o custo dos equipamentos e a limitação da infraestrutura de conectividade no meio rural.

Essa constatação orientou uma decisão de projeto relevante neste trabalho. Em vez de adotar módulos de comunicação sem fio e infraestrutura em nuvem, optou-se por uma arquitetura de operação local, na qual o dado trafega do microcontrolador ao computador do produtor por cabo e é servido por uma aplicação executada na própria rede da propriedade. Trata-se de uma escolha que reduz custo e elimina a dependência de conectividade externa, ao preço de restringir o alcance do acesso remoto — restrição discutida na subseção 6.10.

4.3 Sistemas embarcados e microcontroladores

Sistemas embarcados são sistemas computacionais dedicados a uma função específica, integrados ao dispositivo que controlam e sujeitos a restrições de memória, processamento e consumo energético. Diferentemente de um computador de uso geral, um sistema embarcado executa continuamente um único programa, normalmente estruturado como um laço de repetição que lê entradas, processa e aciona saídas.

O componente central desse tipo de sistema é o microcontrolador, circuito integrado que reúne em um único encapsulamento a unidade de processamento, a memória e as portas de entrada e saída. É essa integração que permite construir dispositivos de sensoriamento com custo e dimensões reduzidos.

4.3.1 A plataforma Arduino

McRoberts (2015) apresenta o Arduino como uma plataforma de computação física de hardware e software abertos, composta por uma placa com microcontrolador e por um ambiente de desenvolvimento que simplifica a programação e a gravação do código. O autor destaca que a plataforma foi concebida para tornar acessível a construção de dispositivos interativos a públicos sem formação prévia em eletrônica ou em programação de baixo nível.

A programação do Arduino é feita em uma variante da linguagem C++ e organiza-se em torno de duas funções obrigatórias. A função de inicialização é executada uma única vez, quando a placa é energizada, e destina-se à configuração dos pinos e dos periféricos. A função de laço é executada indefinidamente enquanto a placa permanecer alimentada, e nela reside a lógica de operação do dispositivo (Arduino, 2026).

Entre os recursos oferecidos pela plataforma, dois foram determinantes para este projeto. O primeiro é a possibilidade de configurar pinos digitais no modo de resistor de elevação interno, que mantém o pino em nível lógico alto sem necessidade de componente externo, invertendo a lógica de leitura: o acionamento do sensor corresponde à queda do sinal para o nível baixo. O segundo é a comunicação serial nativa sobre a mesma porta USB[5] usada para a gravação do programa, o que permite transmitir dados ao computador sem hardware adicional.

4.3.2 Sensores digitais e tratamento de ruído

Sensores digitais entregam ao microcontrolador um sinal binário, correspondente à presença ou à ausência de determinada condição física. Tanto um botão de pressão quanto um módulo sensor infravermelho de barreira se enquadram nessa categoria: ambos fecham ou interrompem um circuito e produzem, na saída, uma transição de nível lógico. Essa equivalência elétrica permite que o mesmo programa opere com qualquer um dos dois componentes, característica explorada no protótipo desenvolvido.

Contatos elétricos, entretanto, não produzem transições limpas. Ao fechar, um contato mecânico oscila por alguns milissegundos antes de estabilizar, fenômeno que faz o microcontrolador registrar múltiplos acionamentos onde houve apenas um. A técnica usual de correção, denominada debounce, consiste em ignorar novas transições do mesmo sensor durante um intervalo mínimo após o primeiro acionamento válido. Sem esse tratamento, um único evento físico geraria diversos registros no sistema, comprometendo a integridade da contagem.

Complementarmente, a detecção por borda distingue o instante da transição do estado contínuo do sinal. Um sensor mantido acionado produz nível baixo constante; o evento de interesse não é esse nível, mas a mudança do nível alto para

[5] [Universal Serial Bus]

o baixo. Comparar o estado atual do pino com o estado lido no ciclo anterior é o mecanismo que isola essa transição.

4.4 Comunicação serial e a Web Serial API[6]

A comunicação serial transmite dados bit a bit por um único canal, em oposição à transmissão paralela. Tanenbaum e Wetherall (2011) situam esse modo de transmissão entre os fundamentos da camada física das redes de computadores, na qual a taxa de sinalização e a sincronização entre emissor e receptor determinam a integridade da comunicação. No caso das placas Arduino, a comunicação com o computador ocorre sobre a porta USB[7], com a taxa de transmissão definida por programa.

Historicamente, o acesso a portas seriais a partir de uma aplicação web exigia um programa intermediário instalado no computador do usuário, responsável por ler a porta e retransmitir os dados. Essa camada adicional representa um custo de instalação e uma fonte recorrente de falhas de configuração, sobretudo quando depende de bibliotecas que exigem compilação de código nativo durante a instalação.

A Web Serial API[8] alterou esse cenário ao permitir que páginas web se comuniquem diretamente com dispositivos seriais conectados ao computador, mediante autorização explícita do usuário (Mozilla, 2026). O navegador apresenta a lista de portas disponíveis, o usuário seleciona o dispositivo e a página passa a ler o fluxo de dados. A mesma fonte registra que o suporte à interface está restrito a navegadores baseados no projeto Chromium, o que constitui uma limitação de compatibilidade a ser considerada no projeto.

A adoção dessa interface neste trabalho eliminou tanto a camada intermediária quanto a configuração manual da porta de comunicação, simplificando de forma significativa a instalação do sistema — decisão detalhada na subseção 6.5.

4.5 Aplicações web e arquitetura cliente-servidor

A arquitetura cliente-servidor organiza um sistema distribuído em dois papéis: o servidor mantém os dados e a lógica de negócio e responde a solicitações; o cliente

[6] [Application Programming Interface (interface de programação de aplicações)]
[7] [Universal Serial Bus]
[8] [Application Programming Interface (interface de programação de aplicações)]

consome esses serviços e apresenta a informação ao usuário. Sommerville (2018) observa que essa separação favorece a manutenção do sistema, pois permite modificar a implementação de um dos lados sem afetar o outro, desde que a interface entre eles seja preservada.

No sistema desenvolvido, essa separação é explícita: o navegador atua como cliente e o servidor de aplicação, escrito em JavaScript e executado no ambiente Node.js, concentra a validação, o armazenamento e a distribuição dos eventos. O mesmo servidor entrega os arquivos estáticos da interface, o que reduz o número de componentes a instalar e configurar.

4.5.1 O estilo arquitetural REST[9]

Fielding (2000) formulou o estilo arquitetural REST[10] a partir da análise dos princípios que sustentam a escalabilidade da web. Entre as restrições que o caracterizam estão a separação entre cliente e servidor, a ausência de estado de sessão no servidor entre requisições, a identificação de recursos por endereços únicos e a utilização de uma interface uniforme de operações.

Na prática, isso se traduz em endereços que identificam recursos e em métodos do protocolo HTTP[11] que expressam a operação pretendida sobre eles. A aplicação desses princípios ao sistema desenvolvido produziu um conjunto reduzido e previsível de endpoints, apresentado na subseção 6.6.2, em que a recuperação de informação é feita pelo método de leitura e a criação de registros pelo método de submissão.

4.5.2 Comunicação bidirecional com WebSocket

O protocolo HTTP[12] opera segundo o modelo de requisição e resposta: o servidor só transmite dados quando solicitado. Esse modelo é inadequado para aplicações que precisam refletir mudanças de estado no instante em que ocorrem, pois obrigaria o cliente a consultar o servidor repetidamente — estratégia que desperdiça recursos e ainda assim introduz latência.

O protocolo WebSocket resolve essa limitação ao estabelecer um canal de comunicação persistente e bidirecional entre cliente e servidor sobre uma única

[9] [Representational State Transfer]
[10] [Representational State Transfer]
[11] [HyperText Transfer Protocol]
[12] [HyperText Transfer Protocol]

conexão, permitindo que o servidor envie dados por iniciativa própria. A biblioteca Socket.IO, adotada neste projeto, implementa esse modelo por meio de eventos nomeados: o servidor emite um evento e todos os clientes conectados que o escutam recebem a mensagem correspondente (Socket.IO, 2026).

Essa característica é o que torna possível o comportamento central do sistema: quando um evento é gravado, todos os navegadores conectados atualizam seus indicadores sem que o usuário precise recarregar a página.

4.6 Persistência de dados

Date (2004) define um banco de dados como uma coleção de dados persistentes, mantida para atender às necessidades de informação de uma organização, e destaca que a escolha do mecanismo de armazenamento deve considerar o volume de dados, a frequência de acesso e os requisitos de integridade e concorrência da aplicação.

Elmasri e Navathe (2018) acrescentam que os sistemas gerenciadores de banco de dados oferecem recursos de controle de concorrência, recuperação de falhas e restrições de integridade cujo custo de adoção só se justifica quando a aplicação efetivamente demanda esses mecanismos. Em sistemas de escopo restrito, com um único processo de escrita e volume reduzido de registros, formatos de arquivo estruturado podem atender aos requisitos de persistência com complexidade substancialmente menor.

O formato JSON[13], especificado pela norma ECMA-404 (Ecma International, 2017), é uma sintaxe textual de intercâmbio de dados estruturados, independente de linguagem de programação e diretamente representável nas estruturas nativas do JavaScript. Essa correspondência direta elimina a necessidade de conversão entre o modelo de dados da aplicação e o formato de armazenamento.

A opção por esse formato no protótipo, em detrimento de um sistema gerenciador relacional, é justificada na subseção 6.6.3 e reconhecida como limitação na subseção 6.10.

4.7 Processo de desenvolvimento por prototipagem

[13] [JavaScript Object Notation]

Pressman e Maxim (2016) descrevem o paradigma de prototipagem como apropriado a situações em que os requisitos não estão inteiramente definidos no início do projeto ou em que a viabilidade técnica de uma solução precisa ser demonstrada antes do comprometimento com uma implementação definitiva. O processo se desenvolve em ciclos curtos de construção e avaliação, nos quais cada versão do protótipo esclarece requisitos para a versão seguinte.

Sommerville (2018) situa a prototipagem entre as abordagens de desenvolvimento incremental e observa que protótipos destinados à demonstração de viabilidade podem legitimamente relaxar requisitos não funcionais — como desempenho sob carga elevada, segurança e robustez operacional — desde que essas simplificações sejam explicitadas.

Esta última observação orientou a condução do presente trabalho. O sistema desenvolvido é um protótipo funcional destinado a demonstrar a viabilidade da integração entre hardware e aplicação web; as simplificações adotadas em relação a um sistema de produção estão registradas ao longo da seção 6 e sistematizadas na subseção 6.10.

5 Materiais e métodos

Esta seção descreve a natureza da pesquisa, as etapas percorridas no desenvolvimento do protótipo, os materiais e recursos empregados e os procedimentos adotados para a validação do sistema.

5.1 Caracterização da pesquisa

Quanto à natureza, esta pesquisa classifica-se como aplicada, uma vez que se destina à geração de conhecimento dirigido à solução de um problema específico e delimitado. Quanto aos objetivos, caracteriza-se como exploratória, na acepção de Gil (2017), por buscar maior familiaridade com um problema com vistas a torná-lo explícito e a construir uma solução para ele.

Quanto aos procedimentos técnicos, a pesquisa combina levantamento bibliográfico e desenvolvimento experimental. O levantamento bibliográfico forneceu os fundamentos conceituais reunidos na seção 4 e subsidiou as decisões de projeto. O desenvolvimento experimental consistiu na construção efetiva do artefato — hardware, firmware, servidor e interface — e na sua submissão a testes funcionais em ambiente controlado.

Quanto à abordagem, o trabalho é predominantemente qualitativo. A validação buscou verificar a correção do comportamento do sistema, isto é, a correspondência entre os eventos físicos provocados e os registros produzidos, e não a mensuração estatística de desempenho sob condições de campo.

5.2 Etapas do desenvolvimento

O desenvolvimento foi conduzido segundo o paradigma de prototipagem descrito por Pressman e Maxim (2016), em ciclos curtos de construção e avaliação. As etapas percorridas, bem como os produtos gerados em cada uma delas, estão sintetizadas no Quadro 1.

Quadro 1 – Etapas do desenvolvimento e respectivos produtos

Etapa	Atividades realizadas	Produto gerado
1. Levantamento	Revisão bibliográfica sobre IoT[14], sistemas embarcados e aplicações web em tempo real; delimitação do escopo do protótipo.	Referencial teórico e definição do escopo.
2. Especificação	Definição dos requisitos funcionais e não funcionais, das regras de negócio e dos casos de uso.	Documento de requisitos (subseção 6.2).
3. Camada física	Definição do mapeamento de pinos, montagem do circuito e implementação do firmware embarcado.	Sketch do microcontrolador.
4. Camada de servidor	Implementação da camada de persistência, da API[15] REST[16] e da difusão por WebSocket.	Servidor Node.js funcional.
5. Camada de apresentação	Construção do dashboard, da página de histórico e da rotina de conexão com o microcontrolador.	Interface web operacional.
6. Integração e testes	Execução do roteiro de testes funcionais e correção das inconsistências identificadas.	Protótipo integrado e validado.

Fonte: Elaborado pelos autores, 2026.

Cabe registrar que as etapas não foram estritamente sequenciais. A construção da camada de apresentação, por exemplo, revelou a necessidade de um endpoint de resumo consolidado que não havia sido previsto na etapa de especificação, o que motivou o retorno à camada de servidor. Esse comportamento é característico do paradigma adotado.

5.3 Materiais e recursos utilizados

5.3.1 Recursos de hardware

O Quadro 2 relaciona os componentes empregados na montagem do protótipo, com a respectiva finalidade.

Quadro 2 – Materiais de hardware utilizados no protótipo

Qtd.	Componente	Finalidade no protótipo
1	Arduino Uno ou Nano	Microcontrolador responsável pela leitura dos sensores e pela transmissão dos eventos.
até 8	Botões de pressão ou sensores IR[17] de barreira (KY-032 / FC-51)	Detecção da passagem do animal na entrada e na saída de cada uma das quatro baias.

[14] [Internet of Things (internet das coisas)]
[15] [Application Programming Interface (interface de programação de aplicações)]
[16] [Representational State Transfer]
[17] [Infrared (infravermelho)]

Qtd.	Componente	Finalidade no protótipo
1	LED[18] indicador	Sinalização visual do registro de cada evento; pode ser substituído pelo LED[19] interno da placa.
1	Resistor de 220 Ω	Limitação de corrente do LED[20] indicador.
1	Buzzer piezoelétrico (opcional)	Sinalização sonora do registro de cada evento.
—	Protoboard e jumpers	Montagem do circuito sem necessidade de solda.
1	Cabo USB[21] de dados	Alimentação da placa e canal de comunicação serial com o computador.

Fonte: Elaborado pelos autores, 2026.

A escolha entre botões de pressão e sensores infravermelhos de barreira não impõe alteração no firmware. Conforme discutido na subseção 4.3.2, ambos os componentes produzem transição do nível lógico alto para o baixo quando acionados, de modo que o programa gravado no microcontrolador opera indistintamente com qualquer um deles. Essa equivalência permitiu conduzir a validação funcional com botões, componentes de menor custo e comportamento mais previsível em bancada, mantendo o caminho aberto para a substituição por sensores de barreira em uma instalação real.

5.3.2 Recursos de software

Os recursos de software utilizados no desenvolvimento e na execução do sistema estão relacionados no Quadro 3, organizados pela camada a que se destinam.

Quadro 3 – Recursos de software utilizados

Camada	Recurso	Versão / observação
Firmware	Linguagem C++ (variante Arduino)	Compilado e gravado pela Arduino IDE[22].
Comunicação	Web Serial API[23]	Interface nativa do navegador; requer Chrome ou Edge.
Servidor	Node.js	Versão 18 ou superior.
Servidor	Express	Versão 4.19 — roteamento HTTP[24] e entrega de arquivos estáticos.

[18] [Light Emitting Diode (diodo emissor de luz)]
[19] [Light Emitting Diode (diodo emissor de luz)]
[20] [Light Emitting Diode (diodo emissor de luz)]
[21] [Universal Serial Bus]
[22] [Integrated Development Environment (ambiente integrado de desenvolvimento)]
[23] [Application Programming Interface (interface de programação de aplicações)]
[24] [HyperText Transfer Protocol]

Camada	Recurso	Versão / observação
Servidor	Socket.IO	Versão 4.7 — comunicação em tempo real.
Servidor	dotenv e CORS	Configuração por variáveis de ambiente e liberação de origem cruzada.
Persistência	Arquivo JSON[25]	Formato conforme a norma ECMA-404.
Interface	HTML5[26], CSS3[27] e JavaScript	Sem utilização de framework de front-end.

Fonte: Elaborado pelos autores, 2026, com base nas dependências declaradas no projeto.

Registra-se que nenhuma das dependências adotadas exige compilação de código nativo durante a instalação. Essa restrição foi estabelecida deliberadamente na etapa de especificação e constitui o requisito não funcional RNF04, apresentado na subseção 6.2.2.

5.4 Procedimentos de validação

A validação do protótipo foi conduzida por meio de testes funcionais de caixa-preta, nos quais se verifica a correspondência entre as entradas fornecidas ao sistema e as saídas por ele produzidas, sem consideração de sua estrutura interna. Sommerville (2018) situa esse tipo de teste entre as práticas de verificação de conformidade do software com sua especificação.

Foram definidos quatro procedimentos de estímulo do sistema, empregados de forma complementar. O primeiro consiste no acionamento físico dos sensores conectados ao microcontrolador, com verificação da saída no monitor serial da Arduino IDE[28]. O segundo utiliza os botões de simulação disponíveis no próprio dashboard, que geram eventos identificados pela origem de demonstração. O terceiro emprega o script gerador de eventos, que submete requisições periódicas ao servidor. O quarto consiste na submissão direta de requisições à API[29] por cliente HTTP[30], utilizado especialmente na verificação das validações de entrada.

Para cada procedimento, verificou-se se o evento foi corretamente persistido, se os totalizadores foram recalculados, se a atualização se propagou às interfaces abertas e se os registros correspondentes apareceram no histórico e na exportação.

[25] [JavaScript Object Notation]
[26] [HyperText Markup Language]
[27] [Cascading Style Sheets]
[28] [Integrated Development Environment (ambiente integrado de desenvolvimento)]
[29] [Application Programming Interface (interface de programação de aplicações)]
[30] [HyperText Transfer Protocol]

O roteiro executado e os resultados obtidos são apresentados na subseção 6.8 e detalhados no Apêndice B.

5.5 Cronograma de execução

O projeto foi executado ao longo de seis meses, conforme o cronograma apresentado no Quadro 4.

Quadro 4 – Cronograma de execução do projeto

Fase	Descrição das atividades	Período
01	Pesquisa e planejamento: levantamento bibliográfico, definição dos requisitos, escolha das tecnologias e divisão de tarefas entre os integrantes.	Meses 1 e 2
02	Desenvolvimento do protótipo físico: montagem do circuito, programação do firmware e verificação da transmissão serial.	Meses 3 e 4
03	Desenvolvimento da aplicação web: implementação do servidor, da camada de persistência e da interface do produtor.	Meses 4 e 5
04	Integração e testes: conexão entre hardware e software, execução do roteiro de testes, correções e validação do sistema completo.	Mês 6

Fonte: Elaborado pelos autores, 2026.

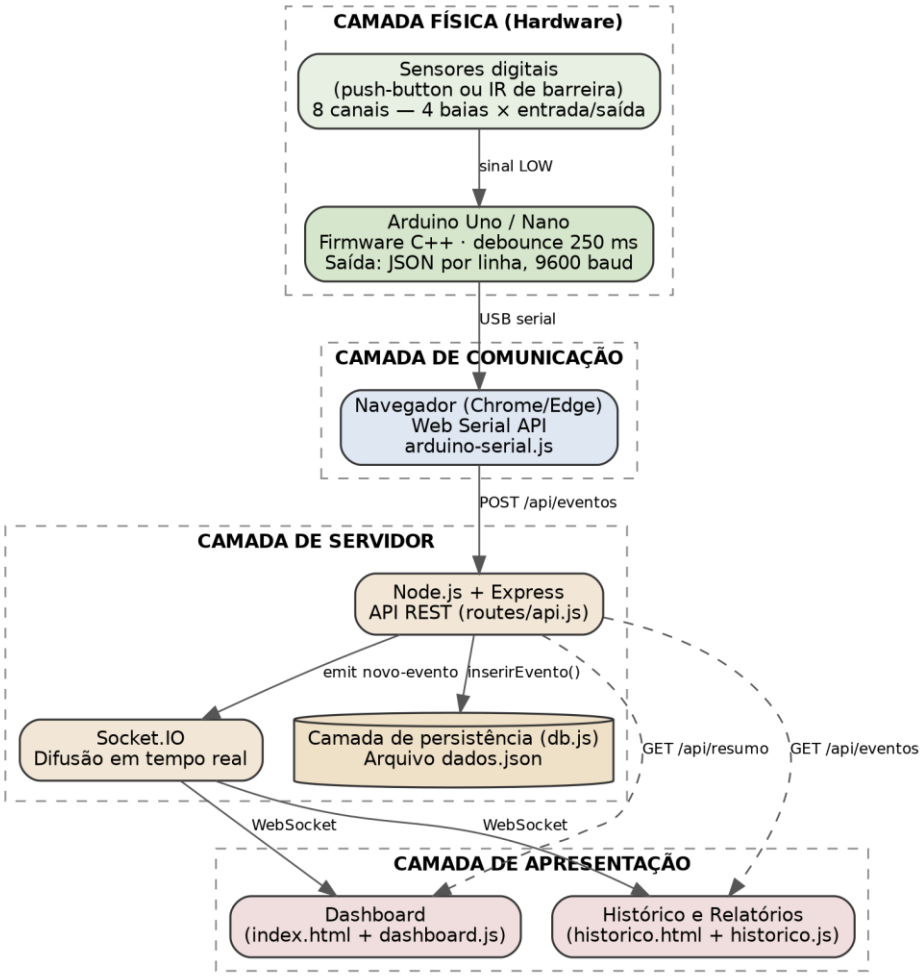
6 Desenvolvimento do sistema e resultados

Esta seção apresenta a construção do protótipo, denominado SuinoTech. A exposição parte da visão arquitetural, avança pela especificação de requisitos e pelos casos de uso e detalha, em seguida, cada uma das camadas implementadas. Conclui com os testes executados, a discussão dos resultados e o registro das limitações identificadas.

6.1 Visão geral e arquitetura da solução

O sistema foi estruturado em quatro camadas, com responsabilidades delimitadas e acopladas por interfaces explícitas. A camada física detecta os eventos no ambiente da granja; a camada de comunicação transporta esses eventos até o servidor; a camada de servidor os valida, persiste e distribui; a camada de apresentação os torna inteligíveis ao produtor. A Figura 1 representa essa organização e os fluxos entre os componentes.

Figura 1 – Arquitetura em camadas do sistema SuinoTech



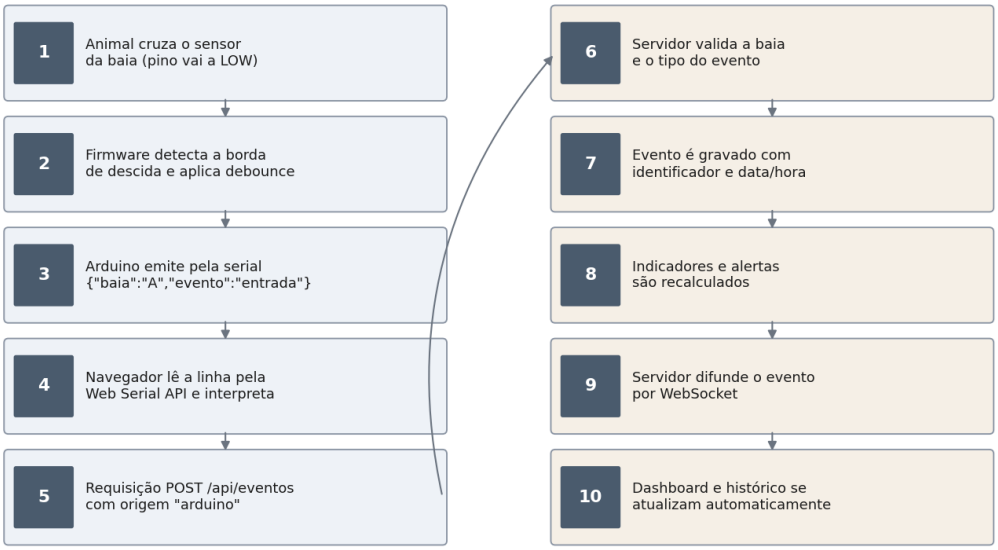
[OBS: FAZER O FLUXOGRAMA NO POWERPOINT]

Fonte: Elaborada pelos autores, 2026.

A separação em camadas atende ao princípio de modularidade discutido por Sommerville (2018): cada camada pode ser substituída sem impacto sobre as demais, desde que a interface seja preservada. Essa propriedade é diretamente relevante para a evolução prevista do sistema, uma vez que a substituição do transporte por cabo pela comunicação sem fio afeta exclusivamente a camada de comunicação, mantendo intactas as camadas de servidor e de apresentação.

O comportamento do sistema pode ser compreendido pelo percurso completo de um evento, desde o acionamento do sensor até a atualização da interface do produtor. A Figura 2 detalha esse percurso.

Figura 2 – Fluxo de um evento, do sensor à interface do produtor



[OBS: FAZER O FLUXOGRAMA NO POWERPOINT]

Fonte: Elaborada pelos autores, 2026.

Observa-se que o servidor não estabelece conexão direta com o microcontrolador. O navegador é o elemento que faz a ponte entre os dois: lê a porta serial, interpreta as linhas recebidas e as reenvia ao servidor como requisições HTTP[31] convencionais. Essa decisão de projeto é justificada na subseção 6.5.

6.2 Especificação de requisitos

Os requisitos foram levantados a partir das limitações identificadas na subseção 1.2 e refinados ao longo dos ciclos de prototipagem. Todos os requisitos aqui relacionados correspondem a funcionalidades efetivamente implementadas e verificadas no protótipo.

6.2.1 Requisitos funcionais

O Quadro 5 apresenta os requisitos funcionais do sistema.

Quadro 5 – Requisitos funcionais do sistema

Código	Descrição
RF01	O sistema deve detectar o acionamento dos sensores de entrada e de saída de cada baia e gerar um evento correspondente.
RF02	O sistema deve registrar, para cada evento, um identificador único, a baia, o tipo (entrada ou saída), a data e a hora do registro e a origem do evento.

[31] [HyperText Transfer Protocol]

Código	Descrição
RF03	O sistema deve calcular e exibir o total de animais presentes em cada baia e no conjunto das instalações.
RF04	O sistema deve exibir o total de entradas e de saídas registradas no dia corrente.
RF05	O sistema deve emitir alerta de atenção quando a ocupação de uma baia atingir 85% de sua capacidade e alerta crítico quando atingir 100%.
RF06	O sistema deve atualizar as informações exibidas nas interfaces abertas assim que um novo evento for registrado, sem intervenção do usuário.
RF07	O sistema deve apresentar o histórico completo dos eventos, ordenado do mais recente para o mais antigo, com paginação.
RF08	O sistema deve permitir a filtragem do histórico por baia, por tipo de evento e por intervalo de datas.
RF09	O sistema deve permitir a exportação do histórico filtrado em arquivo no formato CSV[32].
RF10	O sistema deve permitir ao usuário estabelecer e encerrar a conexão com o microcontrolador a partir da própria interface.
RF11	O sistema deve permitir a geração manual de eventos de entrada e de saída, para fins de demonstração e teste na ausência do hardware.
RF12	O sistema deve exibir a identificação do último evento registrado e a situação da conexão com o microcontrolador.

Fonte: Elaborado pelos autores, 2026.

6.2.2 Requisitos não funcionais

O Quadro 6 apresenta os requisitos não funcionais, que expressam restrições e atributos de qualidade da solução.

Quadro 6 – Requisitos não funcionais do sistema

Código	Descrição
RNF01	A comunicação entre o microcontrolador e o sistema deve ocorrer sem a instalação de programas adicionais no computador do usuário.
RNF02	O sistema deve operar integralmente em rede local, sem dependência de serviços em nuvem ou de conexão com a internet.
RNF03	A interface do produtor deve permanecer funcional mesmo sem o microcontrolador conectado, permitindo a consulta aos dados já registrados.
RNF04	As dependências do servidor não devem exigir compilação de código nativo durante a instalação.
RNF05	O microcontrolador deve descartar acionamentos repetidos do mesmo sensor em intervalo inferior a 250 milissegundos.
RNF06	O sistema deve rejeitar requisições de criação de eventos com baia inexistente ou tipo de evento inválido.
RNF07	O servidor deve entregar os arquivos da interface, dispensando um servidor web adicional.

[32] [Comma-Separated Values]

Código	Descrição
RNF08	A camada de persistência deve expor uma interface estável, de modo que a substituição do mecanismo de armazenamento não afete as demais camadas.

Fonte: Elaborado pelos autores, 2026.

O requisito RNF01 merece registro particular, por ter determinado a escolha da tecnologia de comunicação. Soluções convencionais de leitura de porta serial em ambiente Node.js dependem de bibliotecas que compilam código nativo durante a instalação, exigindo do usuário a presença de ferramentas de compilação no sistema operacional. Em ambiente escolar e em propriedades rurais, essa exigência representa uma barreira concreta de instalação. A adoção da Web Serial API[33], discutida na subseção 6.5, atende simultaneamente aos requisitos RNF01 e RNF04.

6.2.3 Regras de negócio

As regras de negócio implementadas estão relacionadas no Quadro 7. Todas se encontram codificadas na camada de servidor, de modo que sua observância independe do cliente que originou a requisição.

Quadro 7 – Regras de negócio implementadas

Código	Regra
RN01	O sistema reconhece exclusivamente as quatro baias cadastradas, identificadas pelas letras A, B, C e D.
RN02	Um evento pode assumir somente os tipos entrada ou saída; qualquer outro valor é rejeitado.
RN03	O nome da baia informado é normalizado para letra maiúscula antes da validação e do registro.
RN04	O total de animais em uma baia corresponde ao número de entradas menos o número de saídas registradas para aquela baia.
RN05	O total de animais em uma baia nunca assume valor negativo; caso o cálculo resulte em número negativo, o valor apresentado é zero.
RN06	A capacidade padrão de cada baia é de trinta animais.
RN07	Considera-se em nível de atenção a baia cuja ocupação seja igual ou superior a 85% da capacidade e inferior a 100%.
RN08	Considera-se em nível crítico a baia cuja ocupação seja igual ou superior a 100% da capacidade.
RN09	Os totalizadores diários consideram apenas os eventos cuja data coincida com a data corrente do servidor.
RN10	Todo evento registra sua origem, que pode ser o microcontrolador, a inserção manual, a demonstração pela interface ou o script gerador de eventos.

[33] [Application Programming Interface (interface de programação de aplicações)]

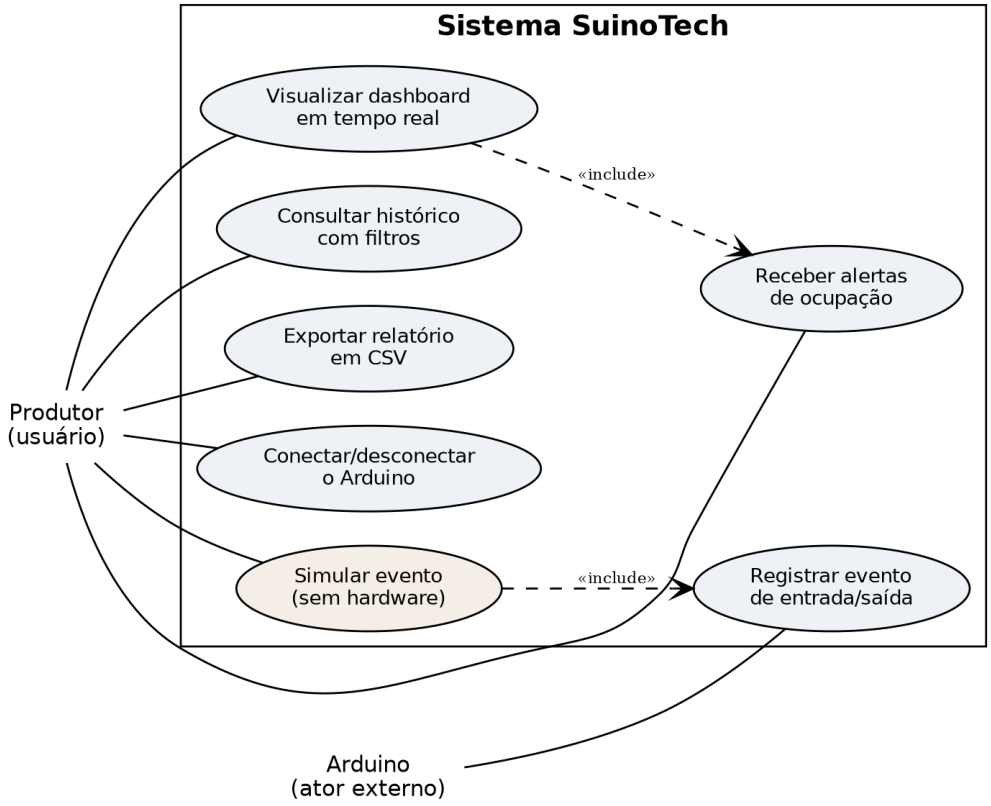
Fonte: Elaborado pelos autores, 2026.

A regra RN05 merece comentário. Como o protótipo não realiza identificação individual dos animais, o saldo de uma baia é obtido por diferença entre contagens. Caso um sensor de saída seja acionado sem que a entrada correspondente tenha sido registrada — situação possível durante testes ou em falha de detecção —, o cálculo produziria valor negativo, desprovido de significado físico. A regra estabelece zero como piso, o que preserva a coerência da informação apresentada, ainda que não corrija a inconsistência de origem. Essa limitação é retomada na subseção 6.10.

6.3 Casos de uso

Os casos de uso do sistema estão representados na Figura 3. Foram identificados dois atores: o produtor, usuário humano da interface, e o microcontrolador, ator externo que origina eventos de forma autônoma.

Figura 3 – Diagrama de casos de uso do sistema



[OBS: FAZER O FLUXOGRAMA NO POWERPOINT]

Fonte: Elaborada pelos autores, 2026, conforme a notação apresentada por Fowler (2005).

O caso de uso de registro de evento é o único compartilhado entre os dois atores: pode ser originado tanto pelo acionamento de um sensor quanto pela ação deliberada do produtor sobre os controles de simulação. Essa convergência decorre de uma decisão de projeto — a de que ambos os caminhos convergem para o mesmo endpoint da API[34] —, o que garante que o evento gerado manualmente percorra exatamente o mesmo tratamento do evento capturado pelo sensor, diferindo apenas no campo que identifica sua origem.

6.4 Camada física: hardware e firmware

6.4.1 Montagem e mapeamento de pinos

O protótipo monitora quatro baias, cada uma com dois sensores digitais — um de entrada e um de saída —, totalizando oito canais de leitura. O mapeamento adotado está descrito no Quadro 8 e representado graficamente na Figura 4.

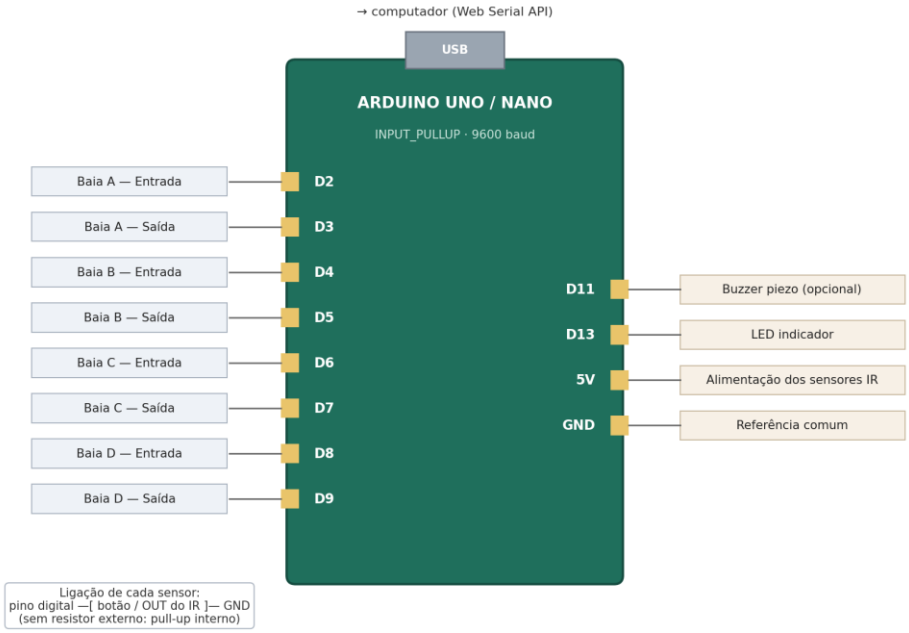
Quadro 8 – Mapeamento de pinos do microcontrolador

Sinal	Pino	Sinal	Pino
Baia A — Entrada	D2	Baia C — Entrada	D6
Baia A — Saída	D3	Baia C — Saída	D7
Baia B — Entrada	D4	Baia D — Entrada	D8
Baia B — Saída	D5	Baia D — Saída	D9
Buzzer (opcional)	D11	LED[35] indicador	D13

Fonte: Elaborado pelos autores, 2026.

Figura 4 – Mapeamento de pinos e esquema de ligação do Arduino

[34] [Application Programming Interface (interface de programação de aplicações)]
[35] [Light Emitting Diode (diodo emissor de luz)]



[OBS: FAZER O FLUXOGRAMA NO POWERPOINT]

Fonte: Elaborada pelos autores, 2026.

A ligação de cada sensor dispensa componentes externos: uma extremidade é conectada ao pino digital correspondente e a outra ao terra do circuito. Essa simplicidade decorre do uso do resistor de elevação interno do microcontrolador, discutido na subseção 4.3.1, que mantém o pino em nível lógico alto na ausência de acionamento. Quando o botão é pressionado — ou quando o feixe do sensor infravermelho é interrompido —, o circuito se fecha com o terra e o pino assume nível baixo, transição que o firmware interpreta como evento.

Figura 5 – Protótipo montado em protoboard

[INSERIR FIGURA — fotografia do circuito montado em protoboard, com o Arduino, os botões correspondentes a pelo menos uma baia, o LED[36] indicador e o cabo USB[37] conectado ao computador. Recomenda-se enquadramento que permita identificar os pinos utilizados.]

Fonte: [PREENCHER — Acervo dos autores, 2026.]

6.4.2 Firmware do microcontrolador

[36] [Light Emitting Diode (diodo emissor de luz)]
[37] [Universal Serial Bus]

O firmware foi implementado na variante da linguagem C++ empregada pela plataforma Arduino. Sua estrutura reflete a organização descrita na subseção 4.3.1: uma rotina de inicialização, executada uma única vez, e um laço principal, executado continuamente.

A rotina de inicialização estabelece a taxa de comunicação serial em 9600 baud, configura os oito pinos de sensor no modo de resistor de elevação interno, define os pinos do LED[38] e do buzzer como saída e inicializa os vetores de controle de estado. Ao final, o microcontrolador emite três pulsos luminosos e transmite uma mensagem de prontidão, que permite ao operador confirmar visualmente e por software que a placa concluiu sua inicialização.

O laço principal percorre as quatro baias e verifica, para cada uma, os sensores de entrada e de saída. A lógica de detecção está concentrada em uma rotina auxiliar, cujo trecho essencial é reproduzido a seguir.

```
int estadoAtual = digitalRead(pino);

// Detecta a borda de descida: estava solto (HIGH) e agora está acionado (LOW)
bool foiAcionadoAgora = (estadoAnterior == HIGH && estadoAtual == LOW);

if (foiAcionadoAgora && (agora - ultimoAcionamento) > DEBOUNCE_MS) {
  enviarEvento(nomeBaia, tipoEvento);
  ultimoAcionamento = agora;
  piscarEBipar();
}

estadoAnterior = estadoAtual;
```

O trecho implementa simultaneamente os dois mecanismos discutidos na subseção 4.3.2. A comparação entre o estado anterior e o estado atual isola a transição do nível alto para o baixo, de modo que um sensor mantido acionado não gera eventos sucessivos. A verificação do intervalo decorrido desde o último acionamento válido, com limiar de 250 milissegundos, descarta as oscilações características do fechamento de contatos mecânicos, atendendo ao requisito RNF05.

Ao confirmar um evento, o firmware transmite pela porta serial uma linha de texto em formato JSON[39] contendo a baia e o tipo do evento, e aciona o LED[40] e o buzzer como confirmação física. A opção pelo formato JSON[41] na saída serial, em lugar de um formato delimitado mais compacto, decorre da correspondência direta

[38] [Light Emitting Diode (diodo emissor de luz)]
[39] [JavaScript Object Notation]
[40] [Light Emitting Diode (diodo emissor de luz)]
[41] [JavaScript Object Notation]

desse formato com as estruturas de dados do JavaScript, o que dispensa rotina de conversão no lado do navegador.

6.5 Camada de comunicação

A comunicação entre o microcontrolador e o servidor constitui o ponto em que este trabalho se afasta da solução convencional. A abordagem usual consistiria em executar, no computador, um processo que abrisse a porta serial e retransmitisse os dados ao servidor. Essa abordagem foi avaliada e descartada por duas razões, ambas ancoradas nos requisitos não funcionais.

A primeira é que as bibliotecas de acesso a porta serial disponíveis para o ambiente Node.js dependem de compilação de código nativo durante a instalação, o que contraria o requisito RNF04 e introduz, na prática, uma barreira de instalação relevante em computadores que não disponham das ferramentas de compilação apropriadas. A segunda é que essa abordagem exige a identificação e a configuração manual da porta de comunicação, procedimento sujeito a erro e sensível à ordem de conexão dos dispositivos.

A solução adotada transfere a leitura da porta serial para o navegador, por meio da Web Serial API[42] descrita na subseção 4.4. A rotina implementada solicita ao usuário a seleção da porta, abre a conexão na taxa de 9600 baud e passa a ler o fluxo de dados de forma contínua, acumulando os caracteres recebidos em um buffer e processando cada linha completa isoladamente.

```
let bufer = "";
while (leituraAtiva) {
  const { value, done } = await leitor.read();
  if (done) break;
  bufer += value;

  let indiceQuebra;
  while ((indiceQuebra = bufer.indexOf("\n")) >= 0) {
    const linha = bufer.slice(0, indiceQuebra).trim();
    bufer = bufer.slice(indiceQuebra + 1);
    if (linha) processarLinha(linha);
  }
}
```

O acúmulo em buffer é necessário porque a leitura de um fluxo serial não garante que cada operação retorne exatamente uma linha completa: um evento pode chegar fragmentado em duas leituras, ou duas linhas podem chegar em uma única

[42] [Application Programming Interface (interface de programação de aplicações)]

leitura. O processamento por delimitador de quebra de linha assegura que cada evento seja interpretado integralmente e uma única vez.

Cada linha válida é convertida em objeto e submetida a duas verificações antes do encaminhamento ao servidor: a mensagem de prontidão emitida na inicialização da placa é descartada, e são ignoradas as linhas cujo conteúdo não corresponda a um par válido de baia e tipo de evento. Essa filtragem impede que mensagens de depuração eventualmente presentes na saída serial sejam interpretadas como eventos.

Confirmada a validade, o evento é encaminhado ao servidor por requisição HTTP[43] ao endpoint de criação, com a origem identificada como proveniente do microcontrolador. A rotina trata ainda o cancelamento da seleção de porta pelo usuário, a perda de conexão durante a leitura e a desconexão física do cabo, atualizando em cada caso o indicador de situação apresentado na interface. Em navegadores sem suporte à interface, o controle de conexão é desabilitado e uma mensagem orienta o usuário, preservando o funcionamento das demais funcionalidades conforme o requisito RNF03.

6.6 Camada de servidor

6.6.1 Estrutura e organização do backend

O servidor foi implementado em JavaScript, sobre o ambiente de execução Node.js, e organizado em três módulos com responsabilidades distintas. O módulo de entrada configura a aplicação, registra as rotas e inicializa o canal de comunicação em tempo real. O módulo de rotas define os endpoints da API[44] e concentra a validação das requisições. O módulo de persistência encapsula todo o acesso aos dados.

Essa separação implementa o princípio de responsabilidade única e materializa o requisito RNF08: nenhum outro módulo acessa diretamente o armazenamento, de modo que a substituição do mecanismo de persistência exige alteração em um único arquivo.

[43] [HyperText Transfer Protocol]
[44] [Application Programming Interface (interface de programação de aplicações)]

O mesmo processo acumula três funções simultâneas: entrega os arquivos estáticos da interface, expõe a API[45] e mantém as conexões em tempo real. A consolidação dessas funções em um único processo atende ao requisito RNF07 e reduz a instalação do sistema a um único comando de inicialização.

6.6.2 A API[46] REST[47]

A interface de programação segue os princípios discutidos na subseção 4.5.1. O Quadro 9 relaciona os endpoints implementados.

Quadro 9 – Endpoints da API[48] REST[49]

Método	Rota	Descrição
GET	/api/resumo	Retorna os indicadores consolidados do painel: total por baia e geral, entradas e saídas do dia, alertas ativos e último evento registrado.
GET	/api/baias	Retorna a relação das baias cadastradas e suas respectivas capacidades.
GET	/api/eventos	Retorna o histórico paginado dos eventos, com filtros opcionais por baia, tipo e intervalo de datas.
GET	/api/eventos/exportar	Retorna o histórico filtrado em formato CSV[50], com cabeçalho de download.
POST	/api/eventos	Registra um novo evento a partir da baia e do tipo informados, e o difunde aos clientes conectados.

Fonte: Elaborado pelos autores, 2026.

O endpoint de criação concentra a validação das entradas, em conformidade com as regras RN01 e RN02. Requisições cuja baia não conste do cadastro ou cujo tipo de evento seja distinto de entrada ou saída são rejeitadas com o código de estado apropriado e uma mensagem que indica os valores aceitos. Confirmada a validade, o evento é persistido, os indicadores são recalculados e o resultado é difundido aos clientes conectados antes que a resposta seja retornada ao solicitante.

Os parâmetros de filtragem do histórico são opcionais e combináveis, o que permite consultas do tipo “saídas da baia B no intervalo de datas especificado” sem que seja necessário prever explicitamente cada combinação. A exportação em CSV[51]

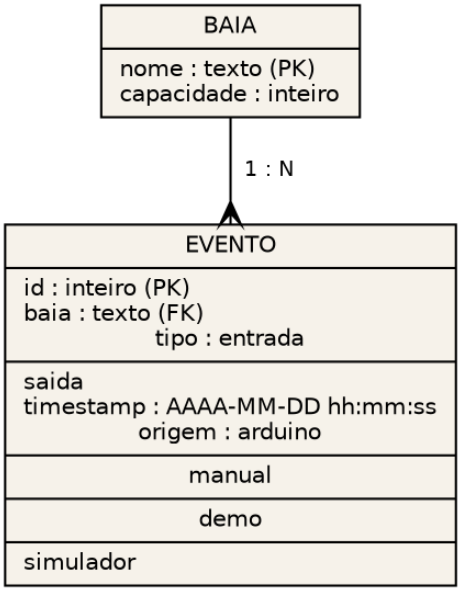
[45] [Application Programming Interface (interface de programação de aplicações)]
[46] [Application Programming Interface (interface de programação de aplicações)]
[47] [Representational State Transfer]
[48] [Application Programming Interface (interface de programação de aplicações)]
[49] [Representational State Transfer]
[50] [Comma-Separated Values]
[51] [Comma-Separated Values]

reutiliza integralmente a mesma rotina de filtragem, garantindo que o arquivo gerado corresponda exatamente ao conjunto de registros exibido na tela.

6.6.3 Persistência e modelo de dados

O modelo de dados do sistema é composto por duas entidades, representadas na Figura 6.

Figura 6 – Modelo de dados do sistema



[OBS: FAZER O FLUXOGRAMA NO POWERPOINT]

Fonte: Elaborada pelos autores, 2026.

A entidade BAIA representa cada unidade monitorada e armazena sua identificação e capacidade máxima, valor utilizado no cálculo dos alertas previstos nas regras RN07 e RN08. A entidade EVENTO representa cada movimentação registrada e mantém identificador sequencial, referência à baia, tipo, data e hora do registro e origem. A relação entre as entidades é de um para muitos: uma baia possui diversos eventos, e cada evento pertence a exatamente uma baia.

A persistência é realizada em arquivo estruturado no formato JSON[52], criado automaticamente na primeira execução do servidor com as quatro baias padrão. Essa escolha, em detrimento de um sistema gerenciador relacional, apoia-se na análise apresentada na subseção 4.6 e em duas considerações objetivas.

[52] [JavaScript Object Notation]

A primeira é de ordem funcional: o protótipo opera com um único processo de escrita e volume reduzido de registros, cenário em que os mecanismos de controle de concorrência e recuperação de falhas oferecidos por um sistema gerenciador — e apontados por Elmasri e Navathe (2018) como sua principal justificativa — não são efetivamente exercitados.

A segunda é de ordem prática e remete ao requisito RNF04: as bibliotecas de acesso a bancos relacionais disponíveis no ecossistema Node.js exigem, em sua maioria, compilação de código nativo durante a instalação, com as mesmas consequências discutidas na subseção 6.5. Optou-se, portanto, por preservar a simplicidade de instalação e isolar a decisão em um único módulo, de modo que a migração para um sistema gerenciador relacional exija apenas a reimplementação interna das funções de acesso, sem alteração de suas assinaturas nem dos módulos que as consomem.

Reconhece-se que essa escolha diverge do que se esperaria de um sistema em produção. Trata-se de uma simplificação deliberada, do tipo que Sommerville (2018) admite em protótipos destinados à demonstração de viabilidade, e está registrada entre as limitações na subseção 6.10.

O cálculo dos indicadores é realizado por agregação sobre a coleção de eventos a cada solicitação, em vez de mantido em contadores persistidos. O trecho a seguir reproduz a lógica de determinação do saldo de uma baia, que implementa as regras RN04 e RN05.

```
function totalAtualPorBaia(nomeBaia) {
  let entradas = 0;
  let saidas = 0;
  for (const e of dados.eventos) {
    if (e.baia !== nomeBaia) continue;
    if (e.tipo === "entrada") entradas++;
    else if (e.tipo === "saida") saidas++;
  }
  return Math.max(entradas - saidas, 0);
}
```

A opção pelo recálculo elimina a possibilidade de divergência entre o total apresentado e o conjunto de eventos que o origina, situação que ocorreria caso um contador persistido deixasse de ser atualizado. O custo dessa garantia é o crescimento linear do tempo de cálculo com o número de registros, aceitável no volume previsto para o protótipo e discutido na subseção 6.10.

6.6.4 Distribuição em tempo real

A atualização automática das interfaces é obtida por meio da biblioteca Socket.IO, conforme os fundamentos apresentados na subseção 4.5.2. O Quadro 10 relaciona os eventos de comunicação implementados.

Quadro 10 – Eventos de comunicação em tempo real

Evento	Sentido	Momento de emissão
resumo-inicial	Servidor → cliente	Imediatamente após o estabelecimento da conexão de um cliente.
novo-evento	Servidor → cliente	Sempre que um evento é persistido com sucesso, difundido a todos os clientes conectados.

Fonte: Elaborado pelos autores, 2026.

O evento de resumo inicial resolve um problema específico de experiência de uso. Como a difusão só ocorre quando um novo registro é criado, um cliente que estabelecesse conexão em um momento sem movimentação permaneceria com a interface vazia até o próximo acionamento de sensor. O envio do estado consolidado no instante da conexão elimina essa espera.

Registra-se que a mensagem difundida transporta não apenas o evento criado, mas o conjunto completo de indicadores recalculados. Essa decisão dispensa o cliente de manter lógica de agregação própria e assegura que todas as interfaces conectadas apresentem valores idênticos, uma vez que todas recebem o resultado do mesmo cálculo executado no servidor.

6.7 Camada de apresentação

A interface foi construída com HTML[53], CSS[54] e JavaScript, sem utilização de framework de front-end. A opção decorre do escopo restrito da aplicação — duas páginas e um conjunto delimitado de elementos dinâmicos — e evita a introdução de uma etapa de compilação no processo de instalação.

6.7.1 Dashboard em tempo real

O painel principal apresenta os indicadores consolidados do plantel. São exibidos o total de animais presentes no conjunto das instalações, a ocupação individual de cada baia com o respectivo percentual, os totais de entradas e saídas do

[53] [HyperText Markup Language]
[54] [Cascading Style Sheets]

dia com a proporção que representam no fluxo diário, os alertas ativos, a identificação do último evento registrado e a situação da conexão com o microcontrolador.

A representação da ocupação emprega codificação por cor derivada dos limiares definidos nas regras RN07 e RN08: a indicação permanece em tom neutro enquanto a ocupação for inferior a 85% da capacidade, passa a tom de advertência no intervalo entre 85% e 100% e assume tom de alerta quando a capacidade é atingida. Essa correspondência permite que a situação das quatro baias seja avaliada sem leitura individual dos valores.

O painel apresenta ainda um gráfico de barras com a distribuição das movimentações nas últimas sete horas, construído por agregação dos eventos em intervalos horários. A janela de observação é recalculada periodicamente, de modo que o gráfico permaneça atualizado mesmo em períodos sem movimentação.

Os controles de conexão com o microcontrolador e de geração manual de eventos estão posicionados no mesmo painel, atendendo aos requisitos RF10 e RF11. Os eventos gerados manualmente são registrados com origem própria, o que permite distingui-los dos eventos capturados por sensor tanto no histórico quanto nos relatórios exportados — distinção relevante para a análise posterior dos dados de teste.

Figura 7 – Interface do dashboard em tempo real

[INSERIR FIGURA — captura de tela da seção de dashboard, com o sistema em operação e dados registrados. É de grande importância que a captura exiba pelo menos um alerta ativo, de modo a evidenciar o comportamento previsto no requisito RF05.]

Fonte: [PREENCHER — Acervo dos autores, 2026.]

6.7.2 Histórico e relatórios

A página de histórico apresenta a relação completa dos eventos registrados, ordenada do mais recente para o mais antigo, em blocos de quinze registros por página. Cada linha exibe o identificador do evento, a baia, o tipo, a data, a hora e a origem do registro.

Os filtros disponíveis — baia, tipo de evento, data inicial e data final — são combináveis e aplicados no servidor, o que evita a transferência do conjunto completo

de registros ao navegador. A exportação em CSV[55], prevista no requisito RF09, respeita os filtros ativos no momento da solicitação, de modo que o arquivo gerado corresponde exatamente ao conjunto consultado.

A página também mantém conexão com o canal de tempo real, mas com comportamento condicionado: a atualização automática só é executada quando o usuário se encontra na primeira página e sem filtros aplicados. Essa condição preserva o contexto de consulta, evitando que uma pesquisa em andamento seja descartada pela chegada de um novo evento.

Figura 8 – Interface de histórico e relatórios

[INSERIR FIGURA — captura de tela da página de histórico, com a tabela de registros preenchida e ao menos um filtro aplicado, evidenciando o total de registros correspondentes ao filtro.]

Fonte: [PREENCHER — Acervo dos autores, 2026.]

6.8 Testes e validação

A validação seguiu os procedimentos descritos na subseção 5.4. O Quadro 11 sintetiza os casos de teste executados e os resultados obtidos. O roteiro detalhado, com os passos de execução de cada caso, encontra-se no Apêndice B.

Quadro 11 – Roteiro de testes funcionais executados

Caso	Objetivo da verificação	Requisito	Resultado
CT01	Verificar a emissão da mensagem de prontidão na inicialização da placa.	RF12	Conforme
CT02	Verificar a geração de evento a partir do acionamento físico do sensor.	RF01	Conforme
CT03	Verificar o descarte de acionamentos repetidos dentro do intervalo de debounce.	RNF05	Conforme
CT04	Verificar o registro completo dos campos do evento.	RF02	Conforme
CT05	Verificar o recálculo do saldo por baia após entradas e saídas.	RF03, RN04	Conforme
CT06	Verificar que o saldo não assume valor negativo.	RN05	Conforme
CT07	Verificar a emissão de alerta de atenção e de alerta crítico nos limiares definidos.	RF05, RN07, RN08	Conforme
CT08	Verificar a atualização automática de duas interfaces abertas simultaneamente.	RF06	Conforme

[55] [Comma-Separated Values]

Caso	Objetivo da verificação	Requisito	Resultado
CT09	Verificar a rejeição de baia inexistente e de tipo de evento inválido.	RNF06, RN01, RN02	Conforme
CT10	Verificar a filtragem do histórico por baia, tipo e intervalo de datas.	RF08	Conforme
CT11	Verificar a correspondência entre o histórico filtrado e o arquivo exportado.	RF09	Conforme
CT12	Verificar o funcionamento da interface com o microcontrolador desconectado.	RNF03	Conforme
CT13	Verificar o comportamento da interface em navegador sem suporte à Web Serial API[56].	RNF01	Conforme, com restrição

Fonte: Elaborado pelos autores, 2026.

O caso CT13 exige esclarecimento. Em navegadores que não implementam a Web Serial API[57], o controle de conexão é corretamente desabilitado e o usuário recebe orientação quanto ao navegador apropriado; as demais funcionalidades permanecem operacionais. O resultado é considerado conforme à especificação, mas registra-se a restrição de compatibilidade dela decorrente, retomada na subseção 6.10.

Ressalta-se que os testes verificaram a correção funcional do sistema em ambiente controlado. Não foram realizados ensaios de desempenho sob volume elevado de registros, nem validação em instalação real de granja, com animais e sensores de barreira em condições de campo. Essas verificações extrapolam o escopo definido para o protótipo e integram as perspectivas de continuidade apresentadas na subseção 7.1.

[VERIFICAR ANTES DA ENTREGA — o Quadro 11 relaciona os casos de teste derivados dos requisitos especificados na subseção 6.2, cujos comportamentos correspondem ao que está implementado no código-fonte. Nós devemos confirmar a execução de cada caso no protótipo montado e registrar, para a banca examinadora, a data de realização do roteiro, o ambiente utilizado e a quantidade de eventos gerados em cada caso. Havendo divergência entre o comportamento observado e o resultado indicado, o quadro deve ser atualizado.]

6.9 Discussão dos resultados

[56] [Application Programming Interface (interface de programação de aplicações)]
[57] [Application Programming Interface (interface de programação de aplicações)]

O resultado principal do trabalho é um protótipo funcional que percorre integralmente o fluxo previsto: o acionamento de um sensor conectado ao microcontrolador produz, sem intervenção humana, um registro persistido e imediatamente refletido nas interfaces abertas. A questão de pesquisa formulada na subseção 1.2 — a viabilidade técnica de construir esse sistema com componentes de baixo custo e tecnologias abertas — é respondida afirmativamente pela existência do artefato e pela conformidade verificada nos testes.

Os objetivos específicos definidos na subseção 3.2 foram atendidos. O levantamento teórico está consolidado na seção 4 e cada decisão de projeto relevante remete explicitamente ao fundamento que a sustenta. A especificação de requisitos e das regras de negócio está registrada na subseção 6.2 e todos os itens relacionados correspondem a comportamentos implementados. O firmware, o servidor e a interface foram implementados e descritos, respectivamente, nas subseções 6.4, 6.6 e 6.7. Os testes funcionais foram executados e seus resultados constam da subseção 6.8.

Merecem discussão específica as três limitações identificadas na subseção 1.2, à luz do que o protótipo efetivamente resolve.

A dependência do registro manual é eliminada no ponto de captura: uma vez acionado o sensor, o registro ocorre sem participação do operador, com data e hora atribuídas pelo próprio sistema. Persiste, contudo, a dependência da correta detecção pelo sensor — o sistema não distingue a ausência de movimentação de uma falha de detecção.

A ausência de histórico consultável é resolvida de forma direta. Todos os eventos são preservados com identificação individual e podem ser recuperados por baia, por tipo e por intervalo de datas, com exportação em formato compatível com planilhas eletrônicas. Esse conjunto atende à função de registro auditável que sustenta iniciativas de rastreabilidade como as descritas na subseção 4.1.

A inexistência de visibilidade imediata é resolvida no âmbito da rede local. Os indicadores são atualizados no instante do registro em todas as interfaces conectadas, e os alertas de ocupação sinalizam automaticamente as baias que se aproximam da capacidade. A visibilidade obtida, entretanto, é local: o requisito RNF02, que dispensa a dependência de conectividade externa, tem como contrapartida a impossibilidade de consulta remota, discutida a seguir.

Cabe uma observação sobre o alcance da contribuição. O protótipo demonstra viabilidade técnica, não benefício produtivo mensurado. Afirmações sobre ganhos de

produtividade, redução de perdas ou melhoria de indicadores zootécnicos exigiriam ensaio comparativo em condições de campo, com instrumentação e período de observação adequados, o que não integrou o escopo deste trabalho.

6.10 Limitações do protótipo

O reconhecimento explícito das limitações constitui condição para a avaliação adequada do que foi construído. Identificam-se as seguintes.

A primeira é a ausência de identificação individual dos animais. O sistema registra eventos de passagem e opera por contagem agregada, o que impede o acompanhamento do histórico de um animal específico. Como consequência, uma falha de detecção não é recuperável: o saldo permanece incorreto até intervenção manual, e a regra RN05 apenas evita a apresentação de valores desprovidos de sentido físico, sem corrigir a causa.

A segunda é a restrição da comunicação ao cabo USB[58]. O microcontrolador precisa permanecer fisicamente conectado ao computador que executa a aplicação, o que limita a distância entre as baias monitoradas e o ponto de operação do sistema.

A terceira é a dependência do navegador. A conexão com o microcontrolador só se estabelece em navegadores baseados no projeto Chromium, e apenas enquanto a página permanecer aberta. O fechamento do navegador interrompe a captura, ainda que o servidor continue em execução.

A quarta é a limitação do mecanismo de persistência. O armazenamento em arquivo único, discutido na subseção 6.6.3, não oferece controle de concorrência nem recuperação de falhas, e o recálculo integral dos indicadores a cada consulta implica crescimento linear do tempo de resposta com o volume acumulado de registros.

A quinta é a ausência de controle de acesso. O sistema não implementa autenticação, de modo que qualquer usuário com acesso à rede local pode consultar os dados e registrar eventos — restrição aceitável em ambiente de protótipo, mas incompatível com uso em produção.

A sexta é a limitação do escopo de validação. Os testes verificaram a correção funcional em ambiente controlado, com acionamento manual dos sensores. Não houve validação em instalação real, com animais, sensores de barreira sujeitos a

[58] [Universal Serial Bus]

poeira e umidade, nem sob o volume de eventos característico de uma granja em operação.

7 Considerações finais

Este trabalho partiu de uma constatação simples: embora a suinocultura brasileira opere em escala industrial e enfrente exigências crescentes de rastreabilidade, o registro da movimentação de animais entre baias ainda é feito manualmente em parte relevante das propriedades. A questão formulada foi a de saber se seria tecnicamente viável automatizar essa captura com componentes de baixo custo e tecnologias abertas.

A resposta obtida é afirmativa. O protótipo desenvolvido percorre integralmente o fluxo proposto: um sensor digital instalado na passagem de uma baia produz, ao ser acionado, um registro persistido com data, hora, baia, tipo e origem, imediatamente refletido no painel do produtor e disponível para consulta e exportação. Os testes funcionais executados confirmaram a correção desse comportamento em todos os casos previstos, com a única restrição registrada quanto à compatibilidade de navegadores.

Do ponto de vista técnico, três decisões de projeto merecem destaque pelo que representam além do resultado imediato. A transferência da leitura serial para o navegador, por meio da Web Serial API[59], eliminou simultaneamente a necessidade de instalação de programa intermediário e de configuração manual de porta, reduzindo a instalação do sistema a um comando único. A separação da lógica de persistência em módulo próprio preservou a possibilidade de substituição do mecanismo de armazenamento sem impacto sobre as demais camadas. A centralização das regras de negócio no servidor assegurou que eventos originados por sensor ou por ação manual recebessem tratamento idêntico.

Do ponto de vista acadêmico, o trabalho articulou em um artefato único conhecimentos de programação embarcada, desenvolvimento web, modelagem de dados, arquitetura de software e teste de sistemas, permitindo verificar como decisões tomadas em uma camada repercutem sobre as demais — percepção difícil de alcançar no estudo isolado de cada componente curricular.

É igualmente importante delimitar o que o trabalho não demonstra. O protótipo estabelece viabilidade técnica, não benefício produtivo. Não foram conduzidos ensaios em condições de campo nem mensurados efeitos sobre indicadores

[59] [Application Programming Interface (interface de programação de aplicações)]

zootécnicos, e as limitações registradas na subseção 6.10 — ausência de identificação individual, restrição da comunicação ao cabo, dependência do navegador, simplicidade do mecanismo de persistência e ausência de controle de acesso — delimitam com clareza a distância entre o artefato construído e um sistema apto ao uso em produção.

Reconhecida essa distância, permanece o resultado central: a demonstração de que a informação sobre a movimentação do plantel pode ser capturada automaticamente e disponibilizada em tempo real com investimento reduzido, sem infraestrutura em nuvem e sem exigir do produtor conhecimento técnico especializado para instalar e operar o sistema. Em um setor no qual a rastreabilidade se consolida como condição de acesso a mercados, soluções com essas características contribuem para que produtores de menor porte participem desse movimento.

7.1 Trabalhos futuros

As limitações identificadas indicam diretamente as frentes de continuidade do projeto.

A substituição dos sensores acionados manualmente por sensores infravermelhos de barreira instalados nas passagens constitui o passo imediato, viabilizado pela equivalência elétrica discutida na subseção 5.3.1, que dispensa alteração no firmware. A validação em instalação real permitiria avaliar o comportamento da detecção sob poeira, umidade e passagem simultânea de animais.

A incorporação de identificação individual por radiofrequência resolveria a limitação mais significativa do protótipo, permitindo associar cada evento a um animal específico e viabilizando tanto a reconstituição do histórico individual quanto a detecção automática de inconsistências na contagem.

A substituição da comunicação por cabo por módulos de transmissão sem fio eliminaria a restrição de distância entre as baias e o ponto de operação. Conforme discutido na subseção 6.1, essa alteração afeta exclusivamente a camada de comunicação, dado que o endpoint de criação de eventos já aceita requisições de qualquer cliente HTTP[60].

A migração da persistência para um sistema gerenciador de banco de dados relacional atenderia aos requisitos de concorrência, integridade e desempenho de

[60] [HyperText Transfer Protocol]

uma instalação em produção, e está prevista na arquitetura pela estabilidade da interface do módulo de persistência, conforme o requisito RNF08.

Por fim, a implementação de controle de acesso com perfis de usuário e a ampliação dos recursos de relatório — incluindo consolidações por período e notificação ativa dos alertas de capacidade — completariam o conjunto de funcionalidades esperado de um sistema destinado ao uso cotidiano.

REFERÊNCIAS

ARDUINO. **Language reference**. Disponível em: https://www.arduino.cc/reference/en/. Acesso em: 24 ago. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023**: informação e documentação: referências: elaboração. 2. ed. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6024**: informação e documentação: numeração progressiva das seções de um documento: apresentação. 2. ed. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6027**: informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6028**: informação e documentação: resumo: apresentação. Rio de Janeiro: ABNT, 2003.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. 3. ed. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE PROTEÍNA ANIMAL. **Relatório anual 2026**. São Paulo: ABPA, 2026. Disponível em: https://abpa-br.org/abpa-relatorio-anual/. Acesso em: 24 ago. 2026.

ATZORI, L.; IERA, A.; MORABITO, G. The Internet of Things: a survey. **Computer Networks**, v. 54, n. 15, p. 2787-2805, 2010.

DATE, C. J. **Introdução a sistemas de bancos de dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

ECMA INTERNATIONAL. **ECMA-404**: the JSON data interchange syntax. 2. ed. Genebra: Ecma International, 2017.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

EMPRESA BRASILEIRA DE PESQUISA AGROPECUÁRIA. **Software inédito no Brasil facilita a gestão de granjas de suínos**. Concórdia: Embrapa Suínos e Aves, 2020. Disponível em: https://www.embrapa.br/busca-de-noticias/-/noticia/55738853/software-inedito-no-brasil-facilita-a-gestao-de-granjas-de-suinos. Acesso em: 24 ago. 2026.

EMPRESA BRASILEIRA DE PESQUISA AGROPECUÁRIA. **Produção de suínos**. Concórdia: Embrapa Suínos e Aves, 2023. Disponível em: https://www.embrapa.br/web/suinos-e-aves. Acesso em: 24 ago. 2026.

EMPRESA BRASILEIRA DE PESQUISA AGROPECUÁRIA. **Robô gera economia de 37 toneladas de ração por ano na suinocultura brasileira**. Concórdia: Embrapa Suínos e Aves, 2024a. Disponível em: https://www.embrapa.br/busca-de-noticias/-/noticia/88808627/robo-gera-economia-de-37-toneladas-de-racao-por-ano-na-suinocultura-brasileira. Acesso em: 24 ago. 2026.

EMPRESA BRASILEIRA DE PESQUISA AGROPECUÁRIA. **Parceria vai gerar protocolos de produção sustentável e de rastreabilidade para suínos e aves**. Concórdia: Embrapa Suínos e Aves, 2024b. Disponível em: https://www.embrapa.br/busca-de-noticias/-/noticia/91599115/parceria-vai-gerar-protocolos-de-producao-sustentavel-e-de-rastreabilidade-para-suinos-e-aves. Acesso em: 24 ago. 2026.

EMPRESA BRASILEIRA DE PESQUISA AGROPECUÁRIA. **Sistema digital amplia rastreabilidade no uso de dejetos suínos como biofertilizantes**. Concórdia: Embrapa Suínos e Aves, 2026. Disponível em: https://www.embrapa.br/busca-de-noticias/-/noticia/113283707/sistema-digital-amplia-rastreabilidade-no-uso-de-dejetos-suinos-como-biofertilizantes. Acesso em: 24 ago. 2026.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Computação) – University of California, Irvine, 2000.

FOWLER, M. **UML essencial**: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

GIL, A. C. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017.

MCROBERTS, M. **Arduino básico**. 2. ed. São Paulo: Novatec, 2015.

MOZILLA. **Web Serial API**. MDN Web Docs, 2026. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Web_Serial_API. Acesso em: 24 ago. 2026.

OPENJS FOUNDATION. **Express**: fast, unopinionated, minimalist web framework for Node.js. Disponível em: https://expressjs.com/. Acesso em: 24 ago. 2026.

OPENJS FOUNDATION. **Node.js**. Disponível em: https://nodejs.org/. Acesso em: 24 ago. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOCKET.IO. **Documentation v4**. Disponível em: https://socket.io/docs/v4/. Acesso em: 24 ago. 2026.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

TANENBAUM, A. S.; WETHERALL, D. **Redes de computadores**. 5. ed. São Paulo: Pearson, 2011.

[VERIFICAR – MUDAR AS DATAS DE ACESSO DAS REFERENCIAS]

APÊNDICE A – Trechos comentados do código-fonte

Este apêndice reúne trechos representativos do código-fonte desenvolvido, organizados por camada. O repositório completo, com a estrutura de diretórios e as instruções de instalação, encontra-se disponível em [PREENCHER — endereço do repositório ou mídia de entrega].

A.1 Firmware — configuração dos pinos e transmissão do evento

A rotina de inicialização configura os oito pinos de sensor no modo de resistor de elevação interno e inicializa os vetores de controle de estado utilizados na detecção por borda.

```
void setup() {
  Serial.begin(9600);

  for (uint8_t i = 0; i < NUM_BAIAS; i++) {
    pinMode(PINOS_ENTRADA[i], INPUT_PULLUP);
    pinMode(PINOS_SAIDA[i], INPUT_PULLUP);
    estadoAnteriorEntrada[i] = HIGH;  // HIGH = solto, graças ao pull-up
    estadoAnteriorSaida[i] = HIGH;
    ultimoAcionamentoEntrada[i] = 0;
    ultimoAcionamentoSaida[i] = 0;
  }
  // ...
  Serial.println("{\"status\":\"pronto\"}");
}
```

A transmissão do evento monta manualmente a linha em formato JSON[61], evitando a inclusão de bibliotecas de serialização que consumiriam memória do microcontrolador.

```
void enviarEvento(const char* baia, const char* tipo) {
  Serial.print("{\"baia\":\"");
  Serial.print(baia);
  Serial.print("\",\"evento\":\"");
  Serial.print(tipo);
  Serial.println("\"}");
}
```

A.2 Servidor — validação e difusão do evento

O endpoint de criação concentra a validação das regras RN01 e RN02 e difunde o resultado aos clientes conectados antes de responder ao solicitante.

```
router.post("/eventos", (req, res) => {
  const { baia, tipo, origem } = req.body || {};

  if (!baia || !db.baiaExiste(baia)) {
```

[61] [JavaScript Object Notation]

```
      return res.status(400).json({
        erro: `Baia inválida. Use uma destas: ${db.BAIAS_VALIDAS.join(", ")}`,
      });
    }
    if (tipo !== "entrada" && tipo !== "saida") {
      return res.status(400).json({ erro: "Tipo inválido. Use 'entrada' ou
'saida'." });
    }

    const evento = db.inserirEvento({ baia, tipo, origem: origem || "manual" });
    const resumo = db.obterResumo();

    io.emit("novo-evento", { evento, resumo });

    res.status(201).json({ evento, resumo });
});
```

A.3 Servidor — cálculo dos alertas de ocupação

O trecho a seguir implementa as regras RN07 e RN08, atribuindo o nível do alerta conforme a proporção entre o total presente e a capacidade da baia.

```
const ocupacao = capacidade > 0 ? totalAtual / capacidade : 0;
if (ocupacao >= 1) {
  alertas.push({
    baia: nome,
    nivel: "critico",
    mensagem: `Baia ${nome} atingiu a capacidade máxima
(${totalAtual}/${capacidade}).`,
  });
} else if (ocupacao >= 0.85) {
  alertas.push({
    baia: nome,
    nivel: "atencao",
    mensagem: `Baia ${nome} próxima da capacidade máxima
(${totalAtual}/${capacidade}).`,
  });
}
```

A.4 Interface — recepção do evento em tempo real

O cliente registra dois ouvintes: um para o estado consolidado enviado na conexão e outro para os eventos difundidos a cada novo registro.

```
if (typeof io === "function") {
  const socket = io(API_BASE || undefined);

  socket.on("resumo-inicial", (resumo) => {
    renderizarResumo(resumo);
  });

  socket.on("novo-evento", ({ resumo }) => {
    renderizarResumo(resumo);
    atualizarMiniChart();
  });
}
```

APÊNDICE B – Roteiro detalhado de testes funcionais

Este apêndice detalha os procedimentos de execução dos casos de teste sintetizados no Quadro 11. Todos os casos foram executados em ambiente controlado, com o servidor em operação local.

CT01 — Mensagem de prontidão

Energizar a placa com o monitor serial aberto a 9600 baud. Verificar a emissão de três pulsos do LED[62] indicador e da linha de estado de prontidão.

CT02 — Geração de evento por sensor

Com a placa conectada pela interface, acionar o sensor de entrada da baia A. Verificar o aparecimento do evento no histórico e o incremento do total da baia.

CT03 — Descarte por debounce

Acionar o mesmo sensor duas vezes em intervalo inferior a 250 milissegundos. Verificar que apenas um evento foi registrado.

CT04 — Integridade dos campos

Consultar o histórico e verificar o preenchimento do identificador, da baia, do tipo, da data, da hora e da origem do evento gerado.

CT05 — Recálculo do saldo

Registrar três entradas e uma saída na baia B. Verificar que o total apresentado corresponde a dois animais.

CT06 — Piso do saldo

Registrar uma saída em baia sem entradas anteriores. Verificar que o total apresentado permanece em zero.

CT07 — Limiares de alerta

Registrar entradas na baia C até atingir 26 animais (86% da capacidade). Verificar a emissão do alerta de atenção. Prosseguir até 30 animais e verificar a transição para alerta crítico.

---

[62] [Light Emitting Diode (diodo emissor de luz)]

CT08 — Atualização simultânea

Abrir o dashboard em duas janelas distintas. Registrar um evento em uma delas e verificar a atualização automática da outra.

CT09 — Rejeição de entradas inválidas

Submeter à API[63] requisições com baia inexistente e com tipo de evento distinto dos previstos. Verificar a rejeição e a mensagem retornada.

CT10 — Filtragem do histórico

Aplicar isoladamente e em combinação os filtros de baia, tipo e intervalo de datas. Verificar a correspondência dos registros apresentados e do total informado.

CT11 — Exportação

Com filtros aplicados, exportar o relatório em CSV[64] e comparar seu conteúdo com os registros exibidos na tela.

CT12 — Operação sem microcontrolador

Encerrar a conexão com a placa e navegar pelo dashboard e pelo histórico. Verificar que os dados já registrados permanecem acessíveis.

CT13 — Navegador sem suporte

Abrir o sistema em navegador que não implemente a Web Serial API[65]. Verificar que o controle de conexão é desabilitado, que a orientação é exibida e que as demais funcionalidades permanecem operacionais.

[VERIFICAR — registrar a data de execução do roteiro e, se possível, anexar capturas de tela que evidenciem os resultados de cada caso.]

[63] [Application Programming Interface (interface de programação de aplicações)]
[64] [Comma-Separated Values]
[65] [Application Programming Interface (interface de programação de aplicações)]